QueryAPIGate HOW-TO GUIDE 60 · ACCESS

Give each team its own workspace

Current as of QueryAPIGate **v0.49.0** · PDF made 4 November 2026

Latest version: anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=60-give-each-team-its-own-workspace

---

Time: 15 minutes. **You'll end up with:** one QueryAPIGate serving several teams, each managing its own connections, saved queries and API keys in a workspace of its own - its owner running it without reaching anyone else's, its apps calling it as if it were the only one, and you, the platform team, seeing all of it.

> **Experimental** - workspaces are new in 0.38 and may change in a minor release, always noted in the changelog (what that means). A server that never creates one behaves exactly as before.

Until 0.38 a server had one namespace: every administrator's role applied to every connection, key and query, so "the payments team runs its own APIs and can't touch the catalogue team's" took a server per team. A **workspace** (ADR 0023) is that boundary inside one server. Everything a server had is in the workspace `default`.

It is delegated administration, not tenant isolation: every workspace runs in the same process, on the same store, and you - an owner of the server - see everything. It keeps teams out of each other's way and reach.

Every command and response below was run against a real server, with the shared key `demo-key`.

Step 1: Create the workspace

As an owner of the server:

```shell
curl -X POST http://127.0.0.1:5000/api/v1/workspaces -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' \
  -d '{"name": "payments", "description": "The payments team"}'
# {"name":"payments","description":"The payments team","created_at":"2026-10-16
20:16:58","created_by":"admin","members":0}
```

Or in the Console: **Access** → **Workspaces** → **New workspace**.

Step 2: Give the team's lead a role in it - and only there

Bea leads payments. She gets no role on the server itself (`"role": null` - say so: leaving `role` out is refused, so it is never by accident) and the role `owner` in her workspace:

```shell
curl -X POST http://127.0.0.1:5000/api/v1/administrators -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' \
  -d '{"name": "bea", "role": null, "email": "bea@corp.com"}'
curl -X PUT http://127.0.0.1:5000/api/v1/workspaces/payments/members/bea -H 'X-API-Key: demo-key' \
  -H 'Content-Type: application/json' -d '{"role": "owner"}'
# {"admin":"bea","role":"owner","created_at":"2026-10-16 20:16:58","added_by":"admin"}
```

Issue her a token as in guide 43 (`POST /api/v1/administrators/bea/tokens`). A role in a workspace is the same four roles, for that workspace only - an owner there manages its members, but not the server's administrators, settings or other workspaces.

Step 3: The lead works in her workspace

Bea's calls name it - `/w/payments/` in front of any route:

```shell
curl http://127.0.0.1:5000/w/payments/api/v1/me -H 'X-API-Key: qagadm_A9NH...'
# {"name": "bea", "role": "owner", "workspace": "payments", "server_wide": false, "workspaces":
{"payments": "owner"}, ...}
```

Without it she is asked to say which:

```shell
curl http://127.0.0.1:5000/api/v1/queries -H 'X-API-Key: qagadm_A9NH...'
# {"error":"You have no role in the workspace 'default' - only in payments: name the workspace you
mean",
#  "code":"workspace_required","workspace":"default","workspaces":["payments"]}
```

In the Console she needs none of this: signed in, she starts in **payments**, and every screen is that workspace's.

She sets up her team's API as anyone would - a connection, a saved query, a key for the checkout app:

```shell
W=http://127.0.0.1:5000/w/payments
curl -X POST $W/api/v1/connections -H 'X-API-Key: qagadm_A9NH...' -H 'Content-Type: application/json' \
  -d '{"name": "orders-db", "db": "sqlite", "database": "/data/payments.db", "active": true}'
curl -X POST $W/api/v1/queries -H 'X-API-Key: qagadm_A9NH...' -H 'Content-Type: application/json' -d '{
  "name": "orders", "description": "All orders", "connection_name": "orders-db", "publish": true,
  "sql": "SELECT id, customer, total FROM orders ORDER BY id"}'
curl -X POST $W/api/v1/api-keys -H 'X-API-Key: qagadm_A9NH...' -H 'Content-Type: application/json' \
  -d '{"name": "checkout-app", "connections": [], "queries": ["orders"]}'
```

Names are the workspace's own: another team can have an `orders` query and an `orders-db` connection too.

Step 4: The team's apps don't need to know

A key belongs to the workspace it was created in, and every call it makes is there - with no prefix:

```shell
curl http://127.0.0.1:5000/q/orders -H 'X-API-Key: sk_YYK...'
# [{"id":1,"customer":"Ana","total":42.5},{"id":2,"customer":"Ben","total":17.0}]
```

Asked for another workspace, it gets the same answer as for one that doesn't exist - it can't learn which there are:

```shell
curl http://127.0.0.1:5000/w/default/q/orders -H 'X-API-Key: sk_YYK...'
# {"error":"No workspace named 'default'","code":"workspace_not_found"}
```

Step 5: What the platform team sees

`default` doesn't have the team's things - `/api/v1/connections` lists none of them - and you reach them by naming the workspace:

```shell
curl http://127.0.0.1:5000/w/payments/api/v1/connections -H 'X-API-Key: demo-key'
# {"items": [{"name": "orders-db", ...}]}
```

The audit log says where each change happened - in the entry, under its hash chain. Bea reads her workspace's; a server-wide role reads all of it, or one workspace's (`?workspace=payments`):

```shell
curl http://127.0.0.1:5000/w/payments/api/v1/audit -H 'X-API-Key: qagadm_A9NH...'
# create_key checkout-app, save_query orders, create_connection orders-db, set_workspace_member,
create_workspace
#   - each with "workspace": "payments"
```

Alerts, run history, live events and usage quotas are the workspace's own too; in `/metrics` its keys and connections are counted as `payments/checkout-app`, `payments/orders-db`.

Step 6: Keep it in git

A configuration file can describe one workspace - `workspace:` at the top - and is applied there only:

```shell
queryapigate config export --workspace payments --kinds connections,queries > payments.yaml
```

```yaml
format: queryapigate-config
format_version: 1
workspace: payments
connections:
- name: orders-db
  db: sqlite
  ...
```

`queryapigate config apply payments.yaml` (or `POST /w/payments/api/v1/config/apply`) - Bea, as its owner, may apply it; sent to any other workspace it is refused. The CLI's other commands take `--workspace` too: `queryapigate export orders --workspace payments --out ...`.

Removing a workspace

Only an empty one - what it owns is never deleted with it - and never `default`:

```shell
curl -X DELETE http://127.0.0.1:5000/api/v1/workspaces/payments -H 'X-API-Key: demo-key'
# {"error":"Workspace 'payments' still has 1 connection, 1 query, 1 API key - delete or move them
first",
#  "code":"workspace_not_empty","contents":{"connections":1,"queries":1,"API keys":1}}
```

Limits, for now

- A **local-folder destination** is only in `default` - the folders exports may write under are the server's. Use an object-storage destination in a workspace.
- **Signed-in app users** (JWT): set `QUERYAPIGATE_JWT_WORKSPACE_CLAIM` to a claim that names each user's workspace, so the token decides it - a user can't reach another team's by changing the `/w/<workspace>/` prefix. Unset, every signed-in user acts in `default`.
- No per-workspace limits yet (each key's own rate limits and quotas work as before), and no mapping of identity-provider groups to memberships.

See Workspaces for the full reference.